Huffman Compression Algorithm: Implementation Summary

Habiba Ayman
Email: s-habiba.ibrahem@zewailcity.edu.eg

***Abstract*—This report presents a summarized implementation of the Huffman compression algorithm, which assigns variable-length binary codes to characters based on frequency. The system supports both static and adaptive modes and includes visualizations for tree construction. Results demonstrate efficiency in text compression through entropy reduction and file size minimization.**

***Index Terms*—Huffman Coding, Data Compression, Entropy, Adaptive Huffman, Tree Encoding**

I. Introduction

Huffman coding is a fundamental lossless compression technique that exploits the frequency distribution of input symbols. It minimizes the average code length by assigning shorter codes to more frequent characters. This implementation supports both static (pre-computed) and adaptive (dynamic) Huffman coding, providing flexibility for offline and real-time applications.

II. System Architecture

The implementation follows a modular, maintainable architecture:

- **Main**: Controls the program execution and manages input/output flow.
- **FileHandler**: Handles file reading and writing operations for plain text and binary data.
- **HuffmanEncoder / Decoder**: Encodes input using Huffman codes and reconstructs original text during decompression.
- **HuffmanTree**: Manages frequency maps, code tables, and tree updates.
- **TreeVisualizer**: Graphically renders static and adaptive Huffman trees.

III. Algorithm Workflow

Compression Steps:

1) Read input text from file.
2) Generate frequency map of characters.
3) Construct the Huffman tree based on frequencies.
4) Generate binary codes for each character.
5) Encode the input and write compressed binary file.

Decompression Steps:

1) Read encoded binary data.
2) Rebuild the Huffman tree (or use adaptive updates).
3) Decode the binary data into original characters.
4) Write decoded output to a file.

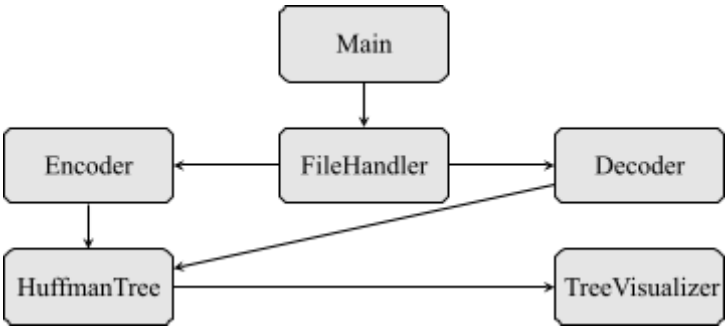
IV. Adaptive Huffman Approach

Adaptive Huffman coding constructs and maintains the Huffman tree incrementally as data is processed:

- **NYT Node (Not Yet Transmitted)**: A placeholder for unseen symbols.
- **Node Weights**: Adjusted dynamically with each new symbol.
- **Tree Swaps**: Ensures tree validity and maintains the sibling property.
- **Single Pass Efficiency**: No need for a separate frequency analysis phase.

V. Tree Visualization

The system visualizes Huffman tree evolution using color-coded nodes and labeled edges. The tree illustrates frequency accumulation and character positioning over time.



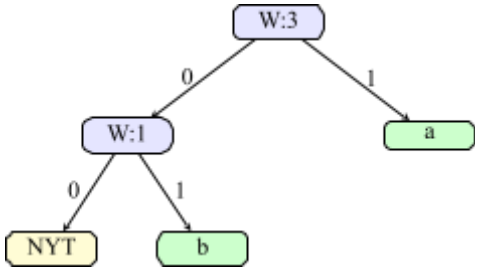


Fig. 1. Adaptive Huffman Tree after encoding “aab”

TABLE I
TEST RESULTS SUMMARY

Input	Bits	Entropy	Pattern
m i s s i s s i p p i	99	2.52	011011010001000...100
aaaaabbbbccccc	93	1.58	a:0, b:10, c:11

VI. EVALUATION AND RESULTS

We tested the implementation on multiple input samples with varying symbol frequencies.

Compression performance improves with higher character frequency skew, demonstrating Huffman's entropy-minimizing capability.

VII. STATIC VS. ADAPTIVE COMPARISON

TABLE II
COMPARISON OF HUFFMAN VARIANTS

Feature	Static Huffman	Adaptive Huffman
Tree Behavior	Fixed	Dynamic
Ideal Use Case	Known stats	Real-time stream
Encoding Speed	Fast	Slower (updates)
Memory Usage	Higher	Lower in stream

VIII. CONCLUSION AND FUTURE WORK

The Huffman compression system successfully reduces file size using frequency-aware binary codes. Its modularity and visual feedback make it educational and practical.